

Запорізький національний університет
Інженерний навчально-науковий інститут ім. Ю.М. Потебні
кафедра електроніки, інформаційних систем та програмного забезпечення

Анотація технології

«Розробка групового проєкту програмного забезпечення на платформі GitHub»

навчально-методичного комплексу дисципліни «Групова динаміка
та комунікації» освітньої програми підготовки бакалаврів
«Програмне забезпечення систем»

Розробник: доцент, к.ф.-м.н. Г.П. Коломoeць

Запоріжжя
2024

Зміст

1.	Теоретичні відомості	3
2.	Попередні вимоги:	4
3.	Інтеграція GitHub у IntelliJ IDEA:	4
4.	Клонування репозиторію - сценарій для кожного учасника команди: .	6
5.	Виконання змін та їх фіксація (коміт)	7
6.	Отримання змін проєкту з віддаленого репозиторію іншим розробником	11
7.	Виявлення конфліктів версій та їх вирішення.	12
8.	Переміщення по історії фіксацій	14
9.	Рекомендований порядок дій при роботі з GitHub	15
	Висновки	16

Мета роботи: Опанування навичок командної розробки програмного забезпечення із використанням системи контролю версій Git та платформи GitHub в середовищі IntelliJ IDEA.

1. Теоретичні відомості

Git — це розподілена система контролю версій, призначена для відстеження змін у файлах та організації командної роботи над програмними проектами.

Ключовими характеристиками Git є:

- розподіленість — кожен розробник має повну копію репозиторію;
- швидкість — більшість операцій виконуються локально;
- цілісність даних — використання хеш-функцій (SHA-1/SHA-256);
- гілкування — ефективна підтримка паралельної розробки.

Git працює з кількома логічними областями (layers) (Рис. 1):

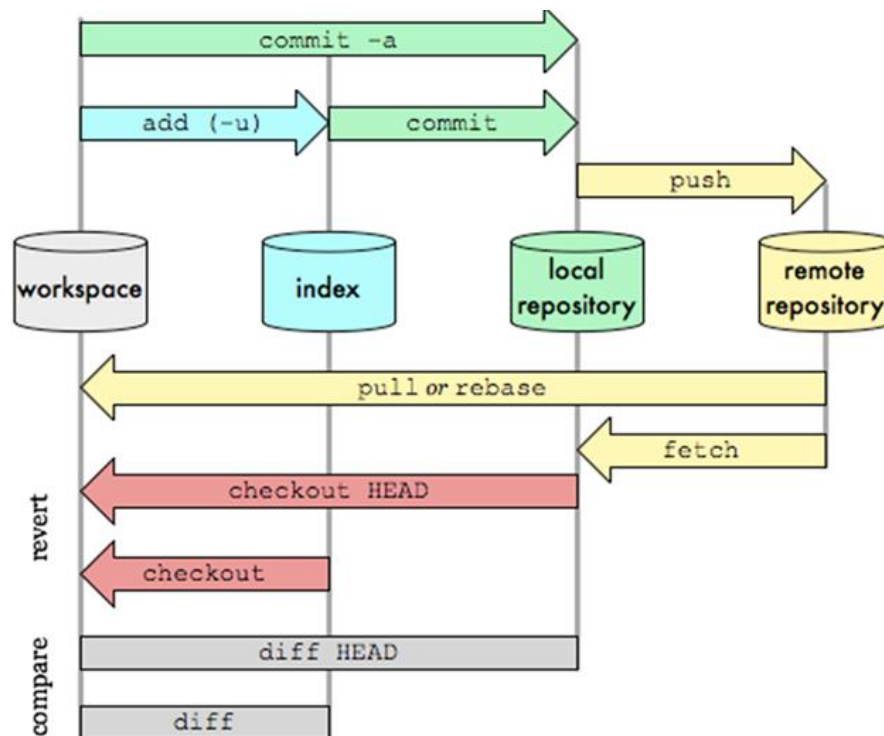


Рис. 1. Етапи робочого процесу Git

- *Робочий каталог (Working Directory/Workspace)* – містить файли проекту на диску Вашого комп'ютера, які змінюються при редагуванні коду;
- *Область індексації (Staging Area / Index)* – це "знімок" (snapshot) стану проекту, який Ви плануєте зафіксувати – дозволяє обирати для фіксації необхідні файли (команда `git add <file>`) та переглядати зміни (команда `git status`) перед остаточним збереженням (комітом – `commit`);
- *Локальний репозиторій (Local Repository)* – це сховище проекту на Вашому комп'ютері, яке зберігає всі файли проекту та повну історію їх змін (у підкаталозі `.git`), дозволяючи працювати офлайн. Підтримує операції фіксації змін (команда `git commit -m "message"`), створення окремих гілок для розробки нових функцій або експериментів

(команда `git branch <name>`), а також їх об'єднання (команда `git merge <name>`), перегляд історії (команда `git log`), порівняння версій (команда `git diff <commit1_id> <commit2_id>`), тимчасове перемикавання на версію в історії (команда `git checkout <commit_id>`) та відкат до попередніх станів (команда `git revert <commit_id>` – якщо зміни вже завантажені у відділений репозиторій (скасовує зміни, створюючи новий коміт-антипод), команда `git reset --hard <commit_id>` – якщо зміни збережені тільки у локальному репозиторії (повертає робочу директорію та історію до вказаного коміту, видаляючи всі наступні коміти)).

- *Віддалений репозиторій (Remote Repository)* – розташований у онлайн системі підтримки командної розробки програмного забезпечення (наприклад, у GitHub або GitLab) та служить центральним сховищем проєкту, забезпечуючи спільну роботу, синхронізацію коду між розробниками, резервне копіювання та хостинг версій. Підтримує операції відправки змін, збережених у локальному репозиторії (команда `git push`), отримання змін, завантажених іншими розробниками (команда `git pull/fetch`) та дозволяє відстежувати віддалені гілки. Основна різниця полягає в тому, що `git pull` завантажує зміни та одразу об'єднує їх (`merge`) з вашою локальною гілкою, тоді як `git fetch` лише завантажує нові дані з віддаленого репозиторію, не змінюючи робочі файли (`pull` – це, по суті, `fetch + merge`).

2. Попередні вимоги:

Студент повинен:

- мати встановлене середовище розробки IntelliJ IDEA;
- мати обліковий запис у GitHub;
- мати доступ до репозиторію з проєктом своєї команди розробників (його організує викладач).

3. Інтеграція GitHub у IntelliJ IDEA:

Для авторизації облікового запису у GitHub у IntelliJ IDEA:

- відкрийте *Settings* (Ctrl+Alt+S для Windows/Linux або ⌘+, для macOS);
- перейдіть до розділу *Version Control* та його підрозділу *GitHub*;
- натисніть кнопку *Add account (+)* (Рис. 2);

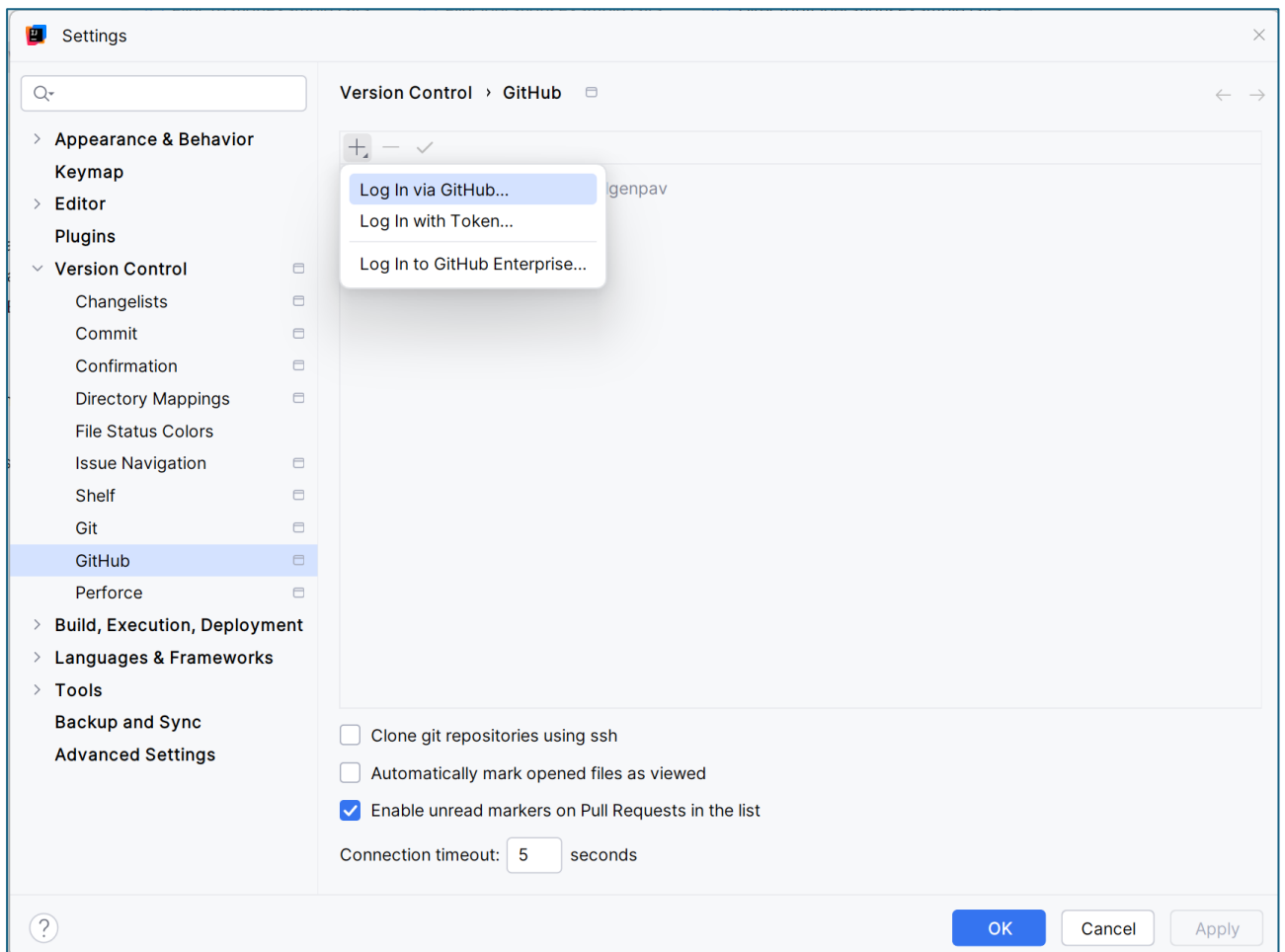


Рис. 2. Додання акаунту GitHub до IntelliJ IDEA

- оберіть команду *Log In via GitHub* – яка відкриє браузер для підтвердження із сторінкою з написом, що акаунт доданий;
- у разі, якщо цей спосіб не спрацює, поверніться на крок назад і оберіть команду *Log In with Token* – у цьому разі відкриється вікно *Add GitHub Account*, у якому потрібно натиснути кнопку *Generate...* (Рис. 3);

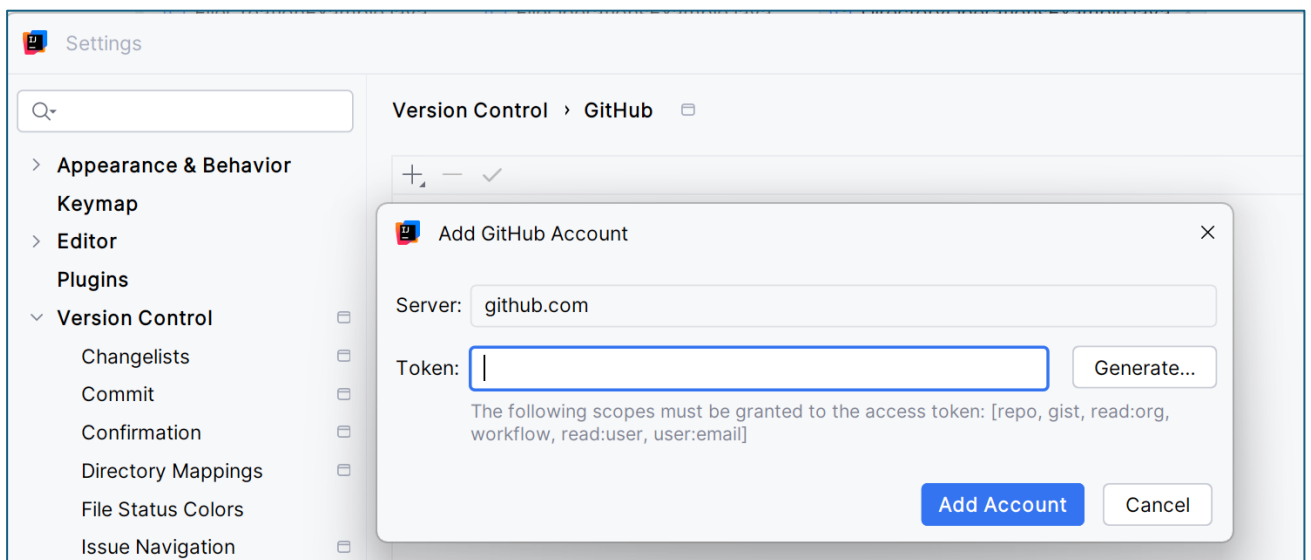


Рис. 3. Генерація та використання токена GitHub

- буде відкрите вікно Вашого профілю GitHub, у якому потрібно обрати термін токена і натиснути кнопку *Generate Token* внизу сторінки;
- буде згенерований токен, який необхідно скопіювати, ввести його у поле *Token* вікна *Add GitHub Account* та натиснути кнопку *Add Account*.

Результатом Ваших дій буде авторизація операцій, виконаних над проєктами GitHub у IntelliJ IDEA, від Вашого імені.

4. Клонування репозиторію - сценарій для кожного учасника команди:

- відкрийте у браузері репозиторій проєкту, над яким працюватиме Ваша команда;
- на сторінці *Code* натисніть кнопку *Code* та скопіюйте адресу репозиторію (Рис. 4)

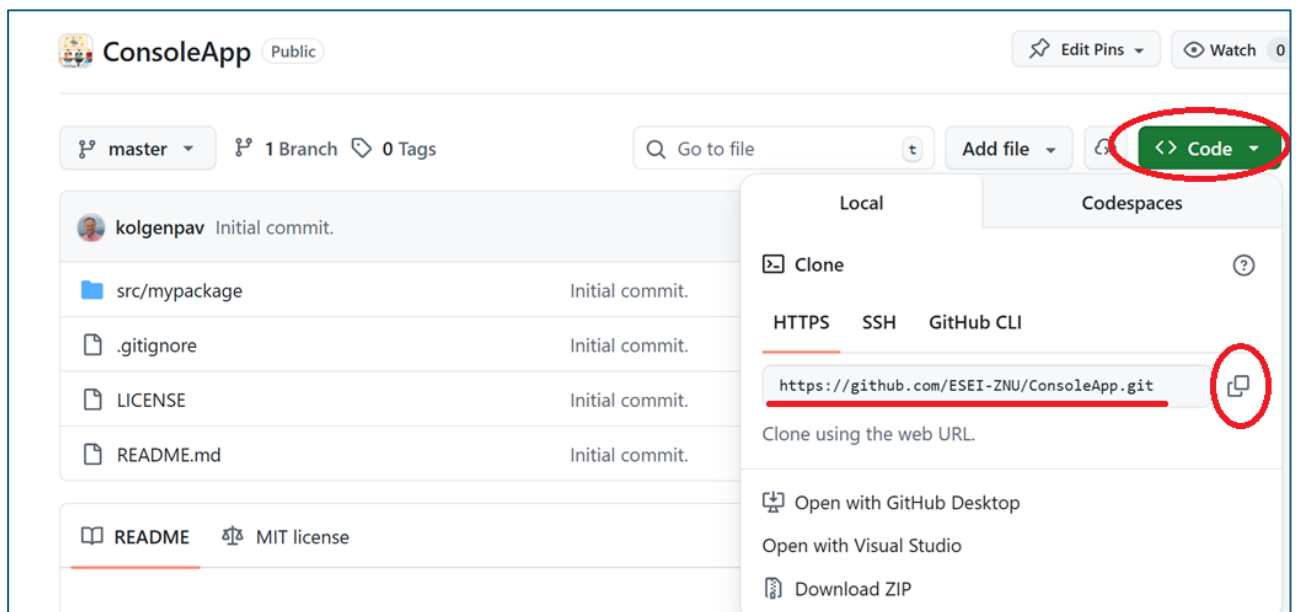


Рис. 4. Отримання адреси репозиторію на GitHub

- відкрийте IntelliJ IDEA;
- виконайте команду *File–New–Project from Version Control...*
- вставте скопійовану адресу репозиторію у поле *URL* вікна *Clone Repository* і натисніть кнопку *Clone* (Рис. 5).

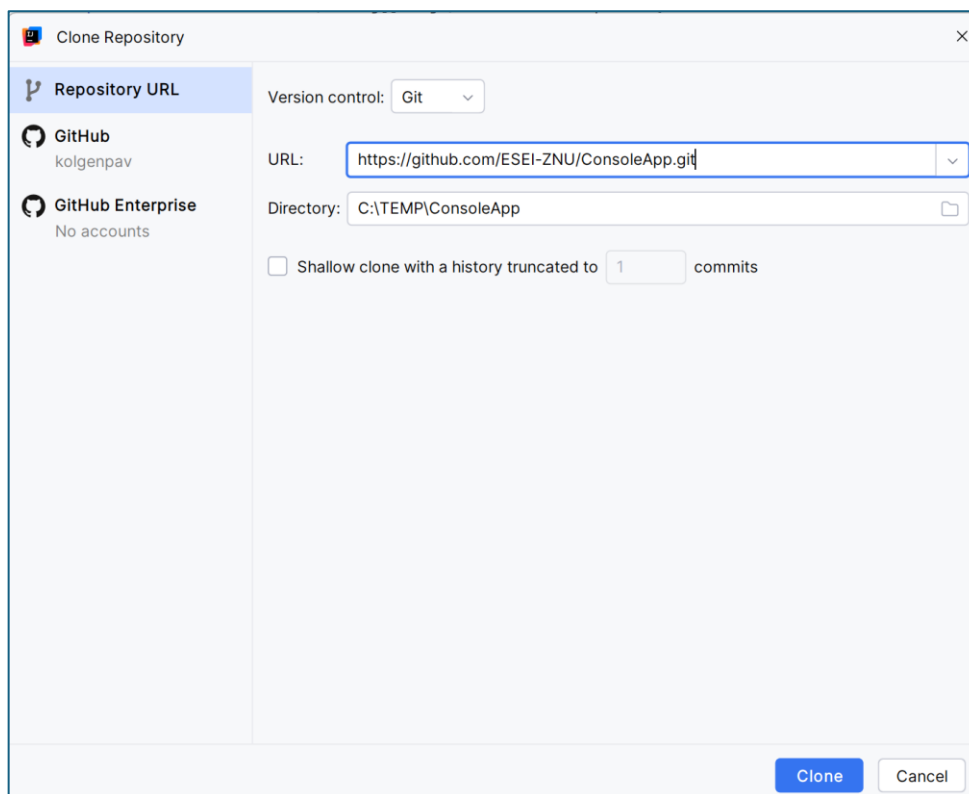


Рис. 5. Зазначення адреси репозиторію на GitHub при клонуванні проєкта

Результатом Ваших дій буде створена локальна копію проєкту, яка зможе синхронізуватися з проєктом у репозиторії GitHub (Рис. 6).

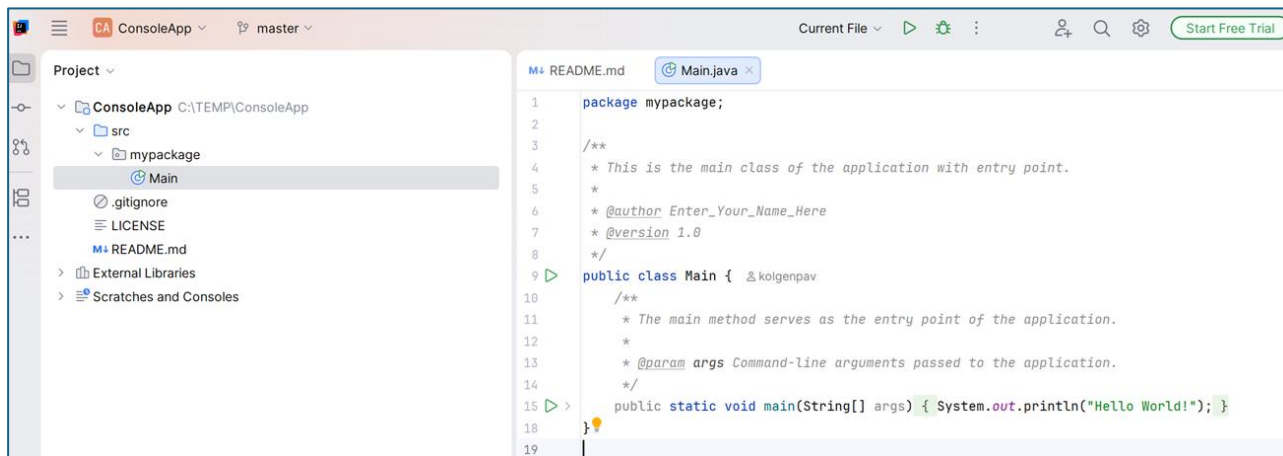


Рис. 6. Зклонований з репозиторію на GitHub проєкт

5. Виконання змін та їх фіксація (коміт)

Після клонування проєкту Ви можете з ним працювати, вносячи необхідні зміни. Нові рядки коду будуть помічені зліва зеленим маркером, а змінені рядки – синім (Рис. 7).

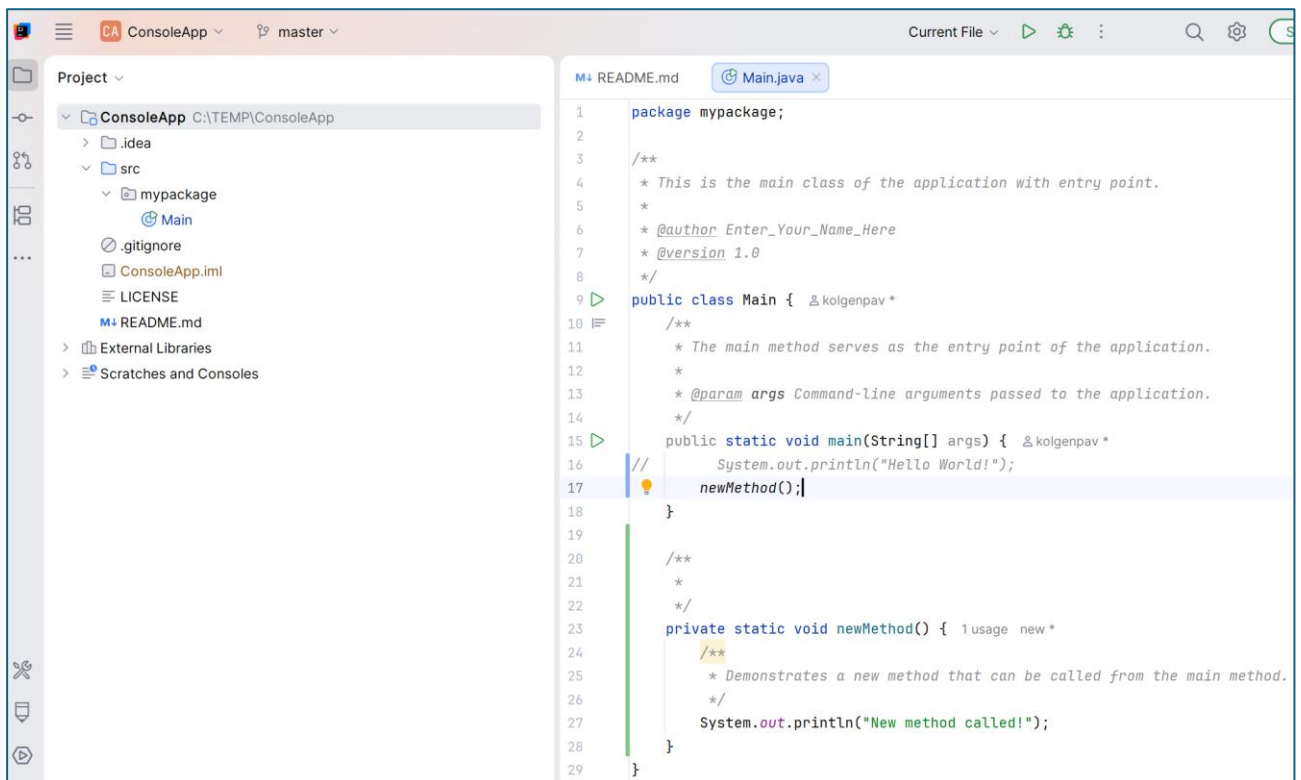


Рис. 7. Маркування змін у синхронізованому з віддаленим репозиторієм проєкті

Після завершення змін можна переходити до їх фіксації (коміта).

Зверніть увагу на те, що рекомендується робити фіксації після завершення певної порції роботи, наприклад, розробки метода, додання коментарів, виправлення помилки тощо. До кожної фіксації можливо повернутися, тому, якщо Ви не будете робити зазначених проміжних фіксацій, а виконаєте єдину фіксацію в кінці роботи над проєктом, то у разі виявлення помилок і необхідності переписати частину коду, це буде неможливо і Вам прийдеться заново писати і правильний і виправлений код!

Для фіксації змін у проєкті необхідно виконати команду головного меню *Git-Commit...* (або клікнути по кнопці  на панелі інструментів зліва). Відкриється вікно *Commit*, у якому у списку *Changes* будуть зазначені файли, що змінені порівняно з попередньою фіксацією (комітом) (Рис. 8).

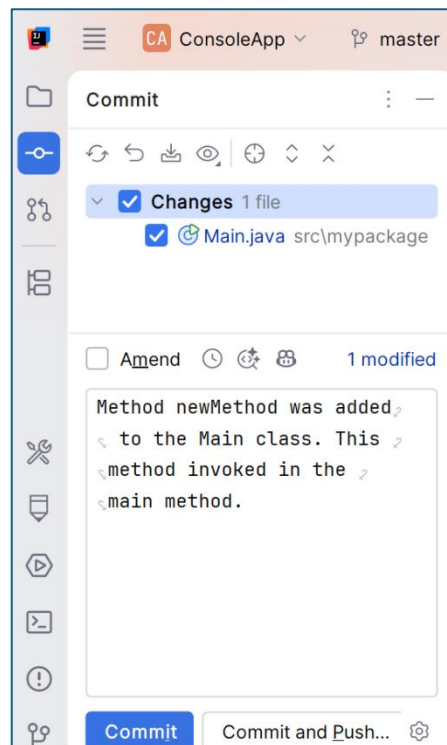


Рис. 8. Підготовка фіксації змін (коміту)

Виконані зміни зручно також переглядати командою *Show Diff* контекстного меню, що відкриється після кліку правою кнопкою миші по зміненому файлу у списку (Рис. 9).

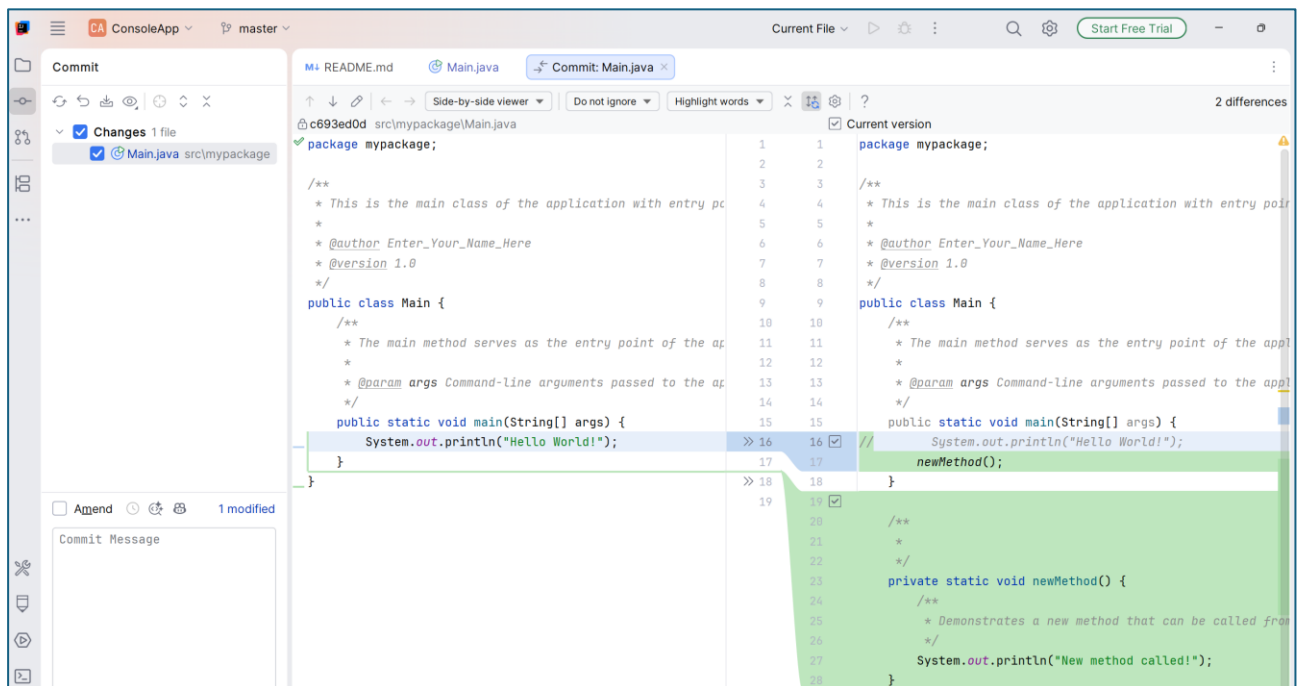


Рис. 9. Перегляд змін порівняно з попередньою зафіксованою версією

Для виконання фіксації (коміту) необхідно у вікні *Commit* ввести текст повідомлення, що буде коротко описувати виконані зміни у цій фіксації, після цього стануть активними кнопки *Commit* та *Commit and Push...*

Кнопка *Commit* виконує фіксацію змін у локальному репозиторії, а кнопка *Commit and Push...* виконує фіксацію змін у локальному та віддаленому репозиторії. Рекомендується використовувати *Commit and Push...*. Якщо буде виводитись вікно з повідомленням, що фіксацію виконати не вдається, це означає, що у коді є недоліки, які побачив внутрішній інспектор якості коду IntelliJ IDEA. Ви можете подивитися на них (та виправити), клікнувши по команді *Review code analysis* (після чого потрібно буде повторно клікнути по кнопці *Commit and Push...*). Можна проігнорувати попередження інспектора якості коду, клікнувши по команді *Commit anyway and push...*.

В результаті відкриється вікно *Push Commits to Назва_проекту*, у якому потрібно клікнути по кнопці *Push* (Рис. 10).

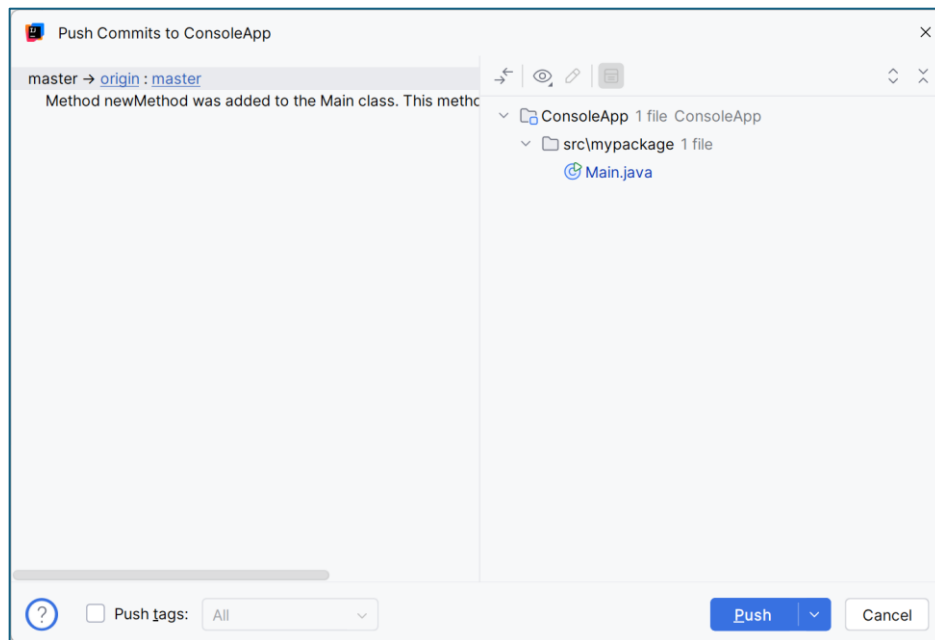


Рис. 10. Фіксація змін у віддаленому репозиторії

Переглянувши репозиторій на GitHub, можна побачити внесені зміни (Рис. 11).

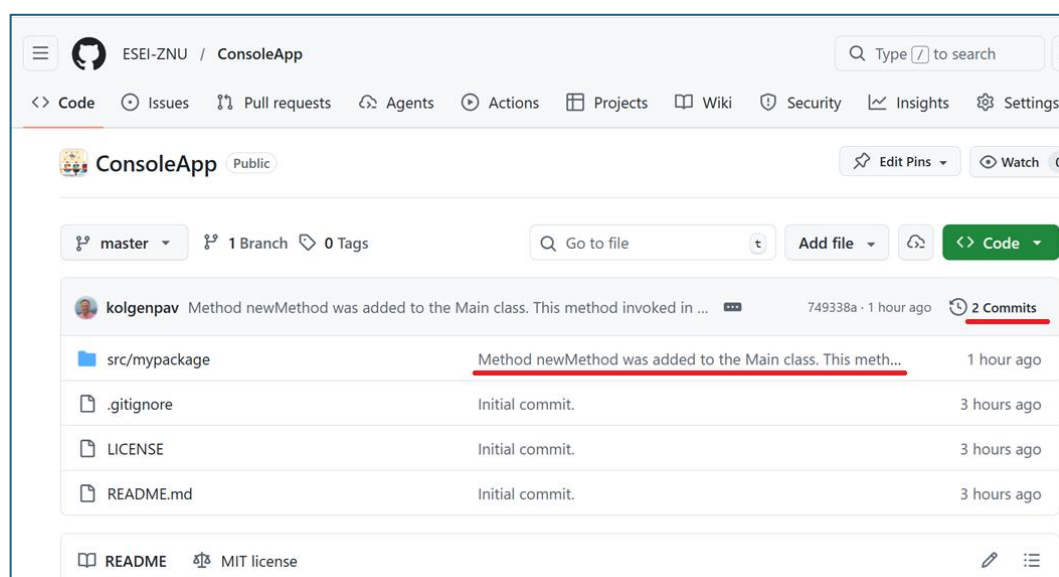


Рис. 11. Перегляд зафіксованих змін у віддаленому репозиторії

6. Отримання змін проєкту з віддаленого репозиторію іншим розробником

Для отримання змін з віддаленого репозиторію, необхідно на початку своєї роботи завжди оновлювати проєкт командою головного меню *Git-Update project...*

Якщо Ви не виконаєте оновлення проєкту, то Ваші зміни будуть конфліктувати з даними у віддаленому репозиторії і буде виводитися повідомлення про помилку!

Після кліку по команді *Git-Update project...* з'явиться вікно *Update project* з опціями оновлення. Рекомендується залишити обрану за замовченням *Merge incoming changes into the current branch* та клікнути по кнопці OK (Рис. 12). В результаті проєкт оновиться до останніх змін у віддаленому репозиторії.

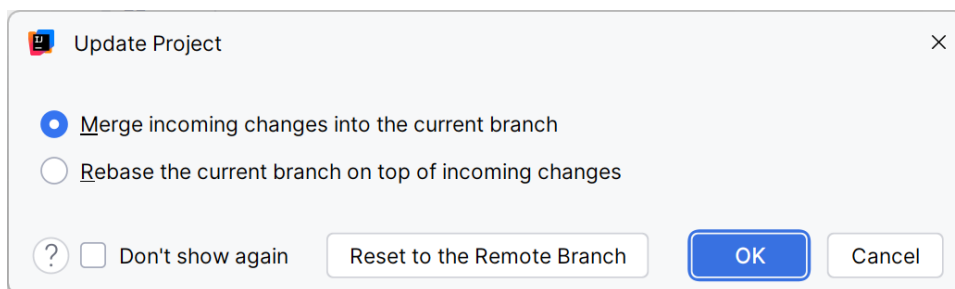


Рис. 12. Вікно опцій оновлення проєкту

У разі додавання нових файлів (наприклад нового класу) до проєкту буде автоматично виведене діалогове вікно з питанням, чи слід додати цей файл до області індексації – рекомендується виконати клік по кнопці *Add* (Рис. 13). Після цього колір шрифту назви класу буде змінений з червоного на зелений, що позначає, що це доданий до області індексації новий клас (червоний колір позначає назви файлів, які Git не обробляє).

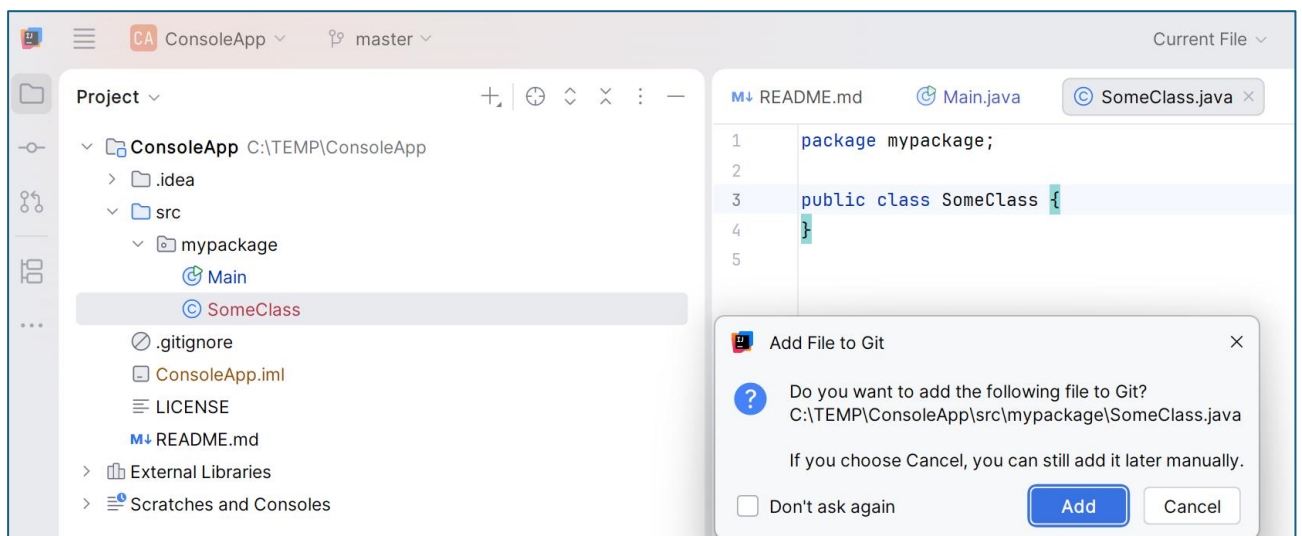


Рис. 13. Вікно додання до області індексації нового файлу

Після виконаних змін другий розробник може виконувати фіксацію (коміт) способом, описаним вище. Можна переглянути зафіксовані зміни у віддаленому репозиторії на GitHub.

7. Виявлення конфліктів версій та їх вирішення.

У разі, якщо декілька розробників внесуть зміни одночасно до того самого класу/методу (Рис. 14) і виконають *Commit and Push*...,

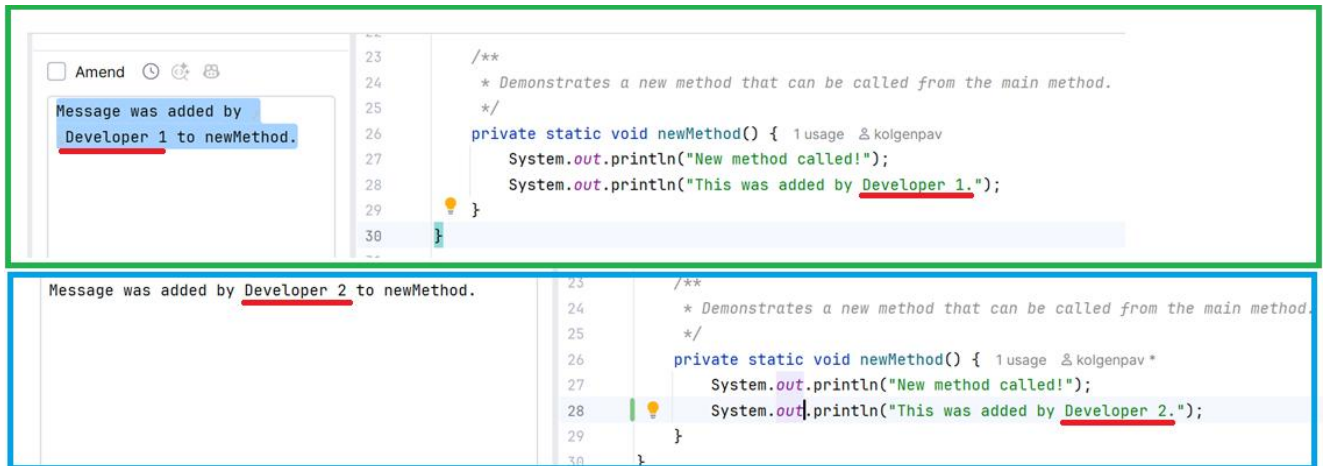


Рис. 14. Створення конфлікту різними розробниками

то для того, хто виконає це пізніше, буде виведене діалогове вікно з повідомленням, що необхідно об'єднувати зміни у віддаленому репозиторії та поточні (Рис. 15).

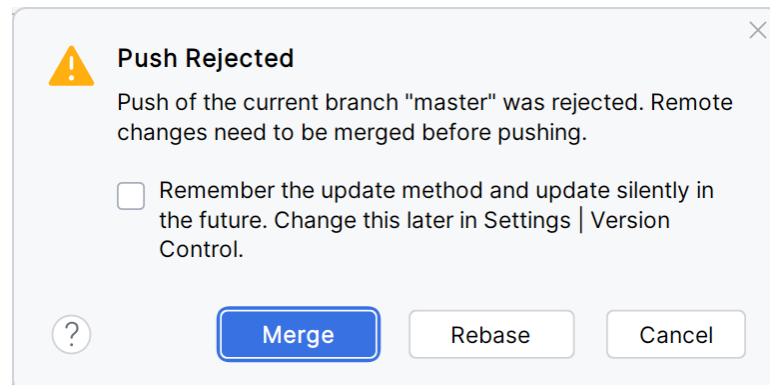


Рис. 15. Вікно відмови у фіксації внаслідок виявленого конфлікту

Рекомендується виконати клік по кнопці *Merge*. Відкриється вікно *Conflicts*, у якому Ви можете побачити назву класу, де є конфлікт версій та кнопки, що дозволяють обрати, які зміни залишити: Ваші – *Accept Yours* або ті, що у віддаленому репозиторії – *Accept Theirs*. Якщо потрібно залишити тільки одну з цих змін, користуйтеся цими кнопками (Рис. 16).

У цьому вікні напроти фрагментів конфлікуючого коду є кнопки, які дозволяють обрати кінцевою або видалити Вашу версію або версію на сервері (на рисунку нижче показане об'єднання змін, внесених обома розробниками) (Рис. 18).

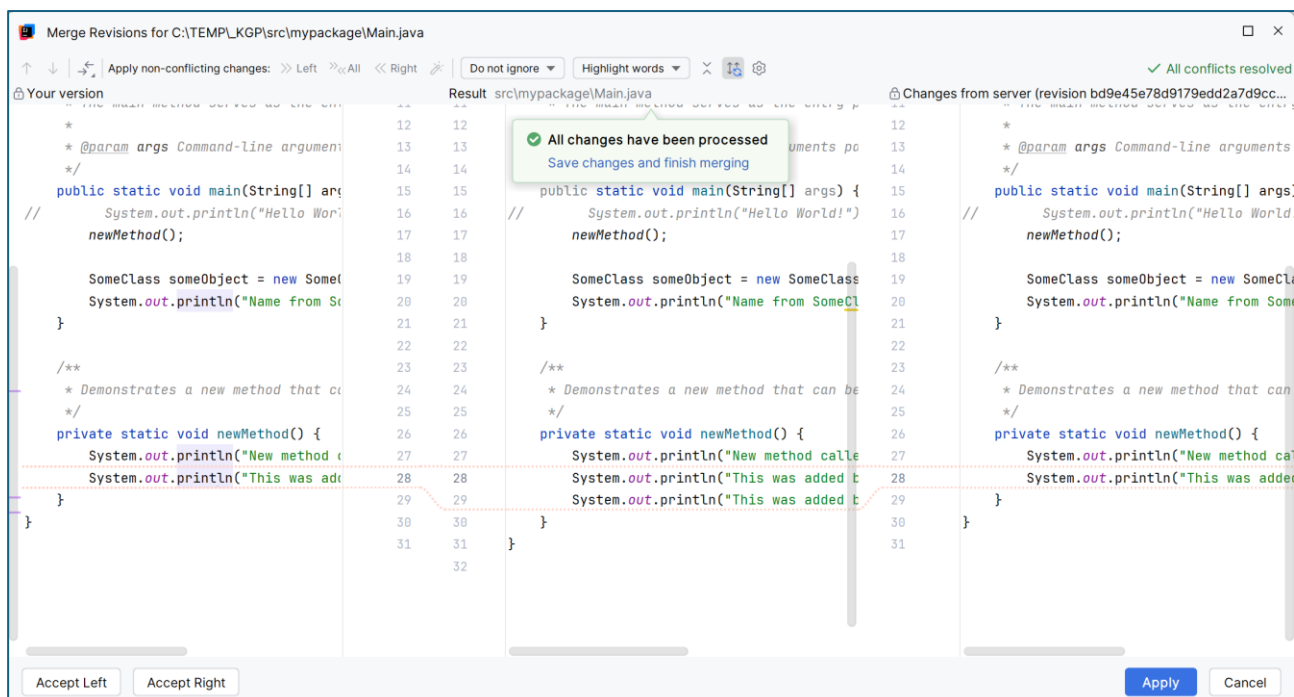


Рис. 18. Вирішення конфлікту шляхом об'єднання змін

Після вирішення усіх конфліктів клікніть по кнопці *Merge* та виконайте фіксацію змін на сервері командою головного меню *Git-Push...*

8. Переміщення по історії фіксацій

Існує команда перегляду історії виконаних фіксацій (комітів). Для її відображення необхідно виконати команду головного меню *Git-Show Git Log*, або

клікнути по іконці  на лівій бічній панелі інструментів IntelliJ IDEA. У нижній частині відкриється вікно з переліком усіх комітів, включаючи ті, що об'єднували зміни декількох розробників (Рис. 19).

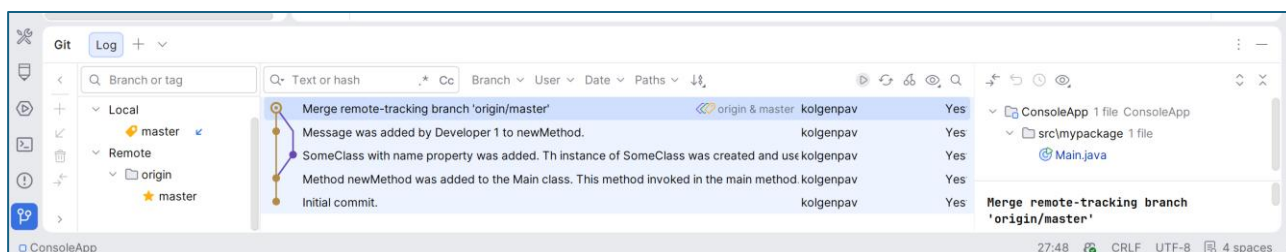


Рис. 19. Журнал фіксацій (комітів)

Якщо потрібно повернути проєкт до стану, у якому він знаходився на момент певної фіксації (коміту), необхідно виконати клік правою кнопкою миші по цій фіксації (коміту) та обрати команду *Checkout Revision* (Рис. 20).

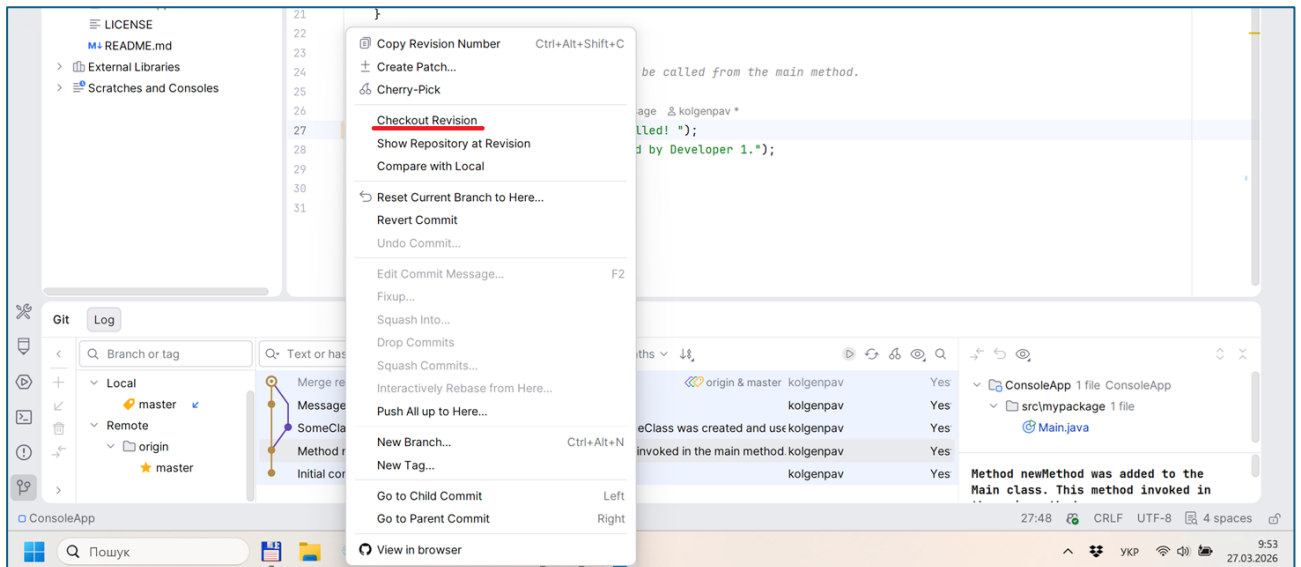


Рис. 20. Повернення стану проєкту до попередньої збереженої фіксації (коміту)

Повернувши проєкт до попереднього стану, Ви можете працювати з ним далі, виконуючи коміти, як було описано вище. Якщо зміни не потрібні, а необхідно було тільки подивитися на стан проєкту, можна повернути стан проєкту до останньої фіксації (коміту). Для цього необхідно клікнути правою кнопкою миші по назві гілки у дереві гілок локального репозиторію у вікні зліва та обрати команду *Checkout* з контекстного меню (Рис. 21).

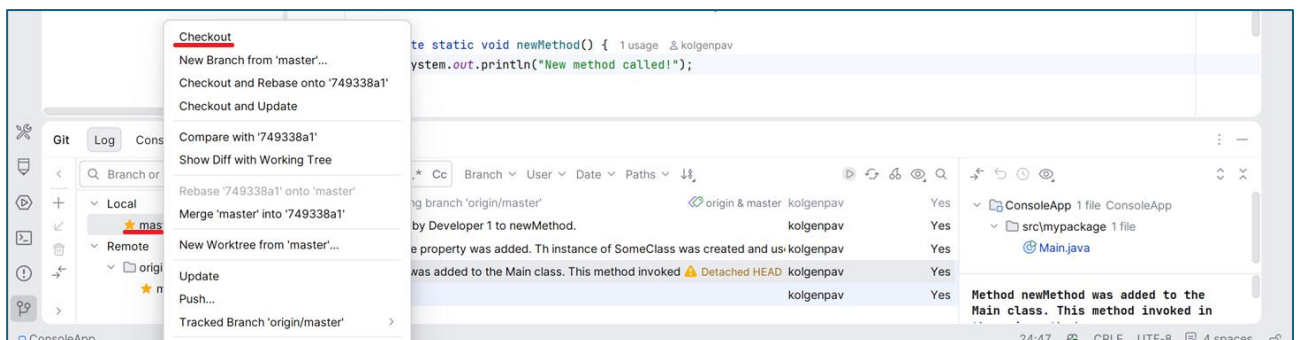


Рис. 21. Повернення стану проєкту останньої збереженої фіксації (коміту)

9. Рекомендований порядок дій при роботі з GitHub

1. Клонування проєкту кожним з розробників.
2. Оновлення проєкту у локальному репозиторії змінами з віддаленого репозиторію на початку сеансу роботи.
3. Виконання фіксації своєї роботи (комітів) у локальному та віддаленому репозиторіях після завершення роботи над кожною частиною свого завдання, додання до комітів коротких, але змістовних коментарів.
4. Вирішення конфліктів у разі їх виявлення при фіксації своєї роботи.
5. Переміщення по історії комітів за необхідності з можливістю зміни виконаної попередньої роботи з обов'язковою фіксацією цих змін.

Висновки

У процесі роботи над командним проєктом студент:

- отримує знання щодо принципів роботи системи контролю версій Git\$
- засвоює базові операції Git;
- опановує інтеграцію GitHub у IntelliJ IDEA;
- набуває навичок використання інструментів IntelliJ IDEA для виконання основних операцій по роботі з Git та GitHub.